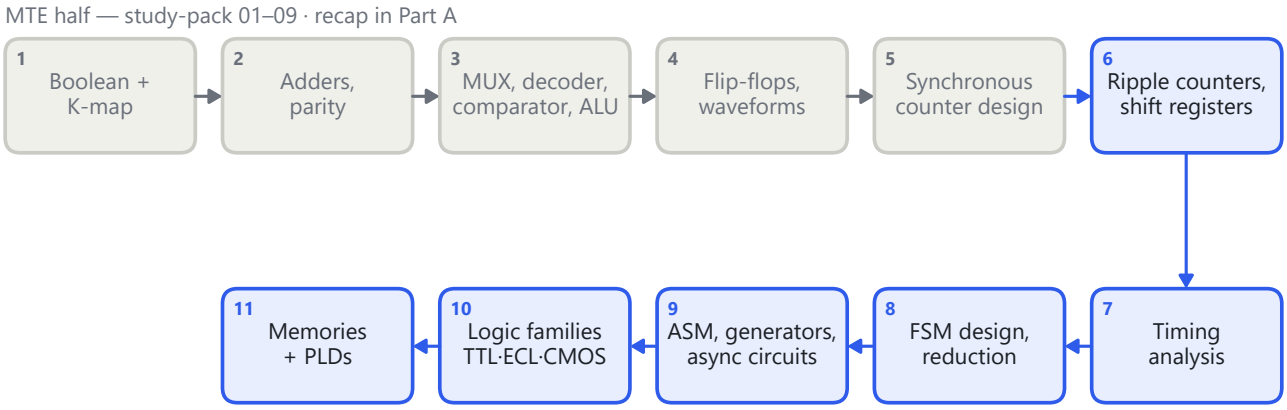
ECE 2105 / ECE2102 · DIGITAL ELECTRONICS · MUJ SEM 3

Digital Electronics: ETE pack, v1

Covers the rest of the syllabus after the MTE: ripple counters, shift registers, timing, state machines, logic families, memories, PLDs. A recap of the MTE half comes first.



Q A real exam question, quoted with its source and marks. If no real paper item was found, it is labelled **practice**.

Guess Write an answer on paper **before** reading on. A wrong guess still helps you remember the answer when you read it.

Check Answer from memory. Answers are at the end of each Part.

Scope comes from the course syllabus slide (deck 1, ECE 2105). Facts come from the prescribed textbooks (Anand Kumar, Jain, Brown & Vranesic, Mano), checked across several sources. Every number in a worked answer is checked by `verify.py`. ETE format is **unknown**, because no ETE paper has been seen. The 2025 MTE used 2-mark recall, 4-mark method and 8-mark design questions, so these pages drill all three.

PART A

The MTE half: recap sheet

Cover the right column and say it out loud, then uncover it. Re-learn any row you missed in study-pack 01–09 tonight. The ETE is comprehensive, so these are still worth marks.

A1 · Minimise and realise

Cue	Recall
K-map group sizes	1, 2, 4, 8 … rectangles, wrap around the edges; each doubling removes one variable.
Don't-cares (X)	Use an X as 1 only when it makes a group bigger. Never chase an X on its own.
POS from a K-map	Group the 0s → get F′ as SOP → complement each term (De Morgan).
NAND-only	Minimal SOP → AND-OR → replace both levels with NAND (two-level NAND–NAND).
NOR-only	Minimal POS → OR-AND → NOR–NOR. Inverter = NOR with its inputs tied together.
Σm ↔ ΠM	ΠM lists exactly the indices that Σm leaves out.

A2 · Arithmetic and parity

Cue	Recall
Half adder	$S = A \oplus B$, $C = AB$
Full adder	$S = A \oplus B \oplus C_{in}$, $C_{out} = AB + C_{in}(A \oplus B)$
Half / full subtractor	$D = A \oplus B$, $B_{out} = A'B$ · full: $D = A \oplus B \oplus B_{in}$, $B_{out} = A'B + B_{in}(A \oplus B)'$
Ripple vs look-ahead	Ripple: the carry crawls through n stages. CLA: $G_i = A_iB_i$, $P_i = A_i \oplus B_i$, $C_{i+1} = G_i + P_iC_i$, expanded so every carry comes out of two gate levels.
BCD adder	If the sum > 9 or there is a carry, add 0110. Trigger $Y = C_{out} + S_3S_2 + S_3S_1$.
Parity	Even-parity bit P = XOR of all the data bits. The checker XORs everything, including P: 0 means no error.

A3 · MSI blocks

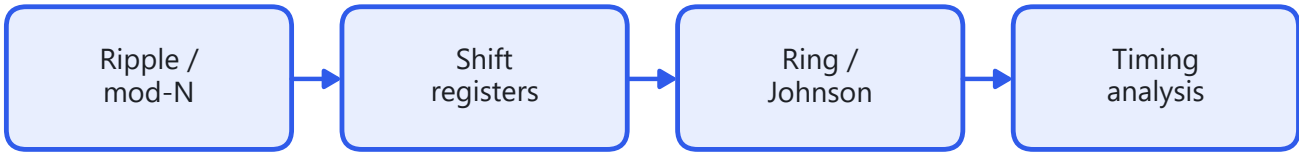
Cue	Recall
n-variable function on a 2^n:1 MUX	Variables go on the selects; data inputs = the truth-table output column.
On a 2^{n-1}:1 MUX	n−1 variables on the selects; each data input is 0, 1, x or x′ (x = the leftover variable).
Decoder + gate	An n:2^n decoder outputs every minterm, so OR the minterms of F. With an active-low decoder, use NAND.
Priority encoder	Outputs the code of the highest-priority active input, plus a "valid" flag.
Comparator (1 bit)	A>B: AB′ · A<B: A′B · A=B: (A ⊕ B)′. For n bits, work from the MSB down using the equality terms.
7-segment, common anode	A segment lights on a LOW, so the decoder outputs are active-low.

A4 · Flip-flops and synchronous counter design

Cue	Recall
Characteristic equations	SR: $Q^+ = S + R'Q$ (SR = 0) · D: $Q^+ = D$ · JK: $Q^+ = JQ' + K'Q$ · T: $Q^+ = T \oplus Q$
Excitation (Q → Q^+)	0→0: JK 0X, T 0, D 0 · 0→1: JK 1X, T 1, D 1 · 1→0: JK X1, T 1, D 0 · 1→1: JK X0, T 0, D 1
Race-around	Level-triggered JK with J = K = 1 toggles many times while CLK is high. Fixes: master–slave or edge-triggered.
Counter design	State table → excitation for each FF → K-map each input (unused states = X) → circuit → check that unused states lead back into the count.

PART B

Sequential logic: the rest of Unit 3



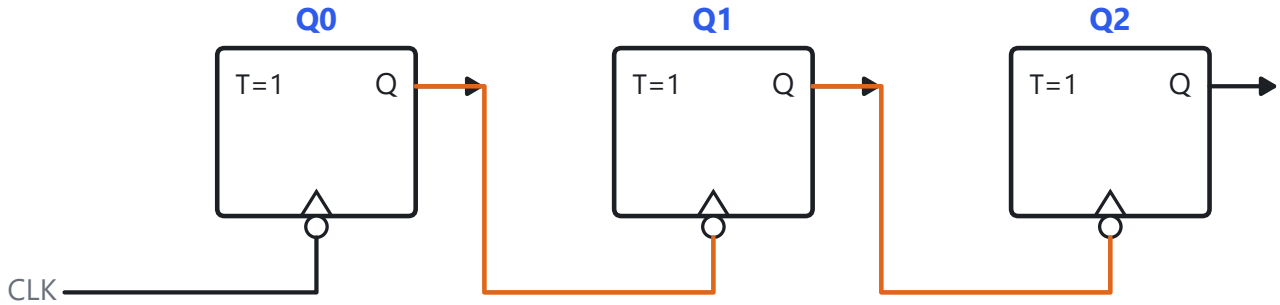
UNIT 3 › COUNTERS

B1 · Ripple (asynchronous) counters

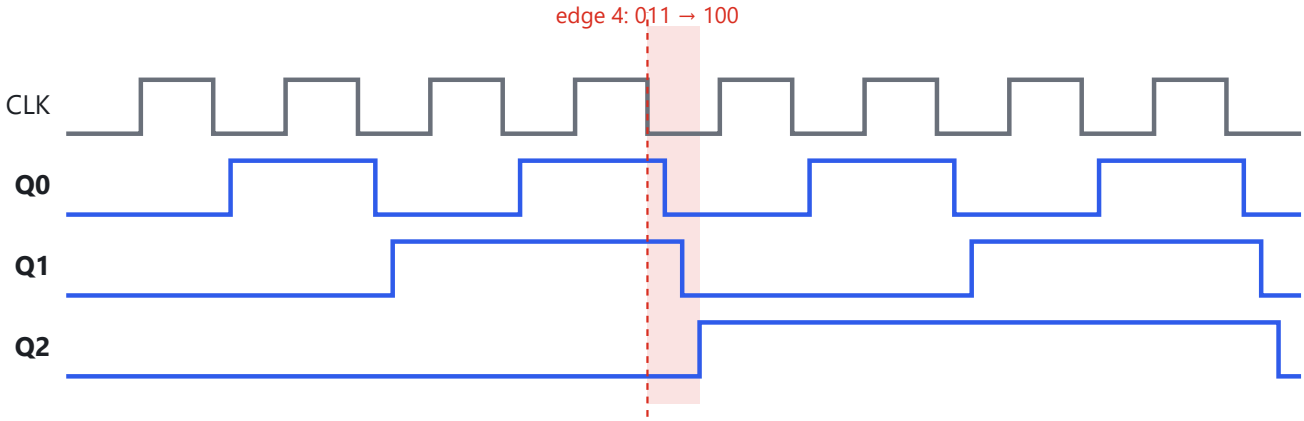
Anna Univ. EC6302 · Apr/May 2015 · Part A Q6 · 2 marks

What is the minimum no of flip flops needed to design a counter of modulus 60?

✎ **Guess first, on paper.**



each flip-flop is clocked by the Q of the one before (negative edge, ○ = falling-edge input)



How it counts. Every FF has T = 1, so it toggles. FF0 toggles on each falling CLK edge. FF1 toggles whenever Q0 falls (1→0), and FF2 whenever Q1 falls. Reading Q2Q1Q0 gives 0, 1, 2 … 7, 0. That is an **up** counter. Clock the next stage from Q′ instead and it counts **down**. With positive-edge FFs the rule flips.

The flaw: ripple delay. Each stage waits for the one before it, so after edge 4 (011 → 100) the outputs pass through 010 and 000 before settling (shaded). The worst-case settle time is $n \cdot t_{pd}$. A decoder reading the outputs sees **glitches**, and $f_{max} \approx 1 / (n \cdot t_{pd})$ drops as n grows. Synchronous counters fix this because every FF shares one clock.

Mod-N. n flip-flops give at most 2^n states, so you need the smallest n with $2^n \geq N$. To stop a ripple counter at N, detect the state N with a gate and use it to drive the asynchronous CLEAR. Mod-10: NAND(Q_3, Q_1) → CLR′ at

1010. Count 1010 appears for a few nanoseconds and is then cleared, which is another glitch.

> **Answer.** $2^5 = 32 < 60 \le 64 = 2^6$, so **6 flip-flops**.

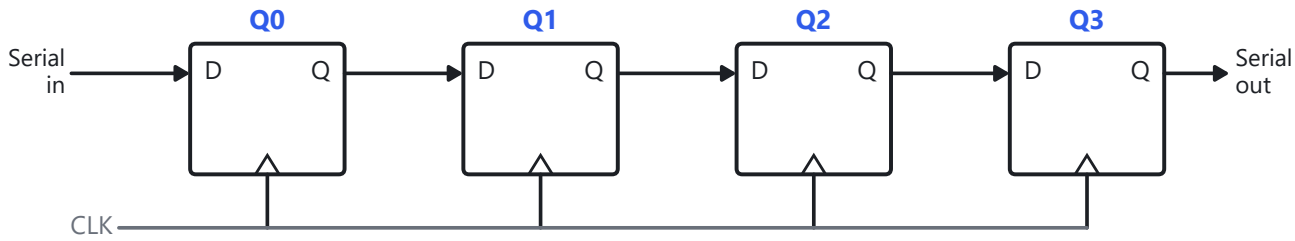
Check B1. (a) A 4-bit ripple counter uses FFs with t_{pd} = 25 ns. Find the longest time it takes to settle after an edge. (b) Which gate inputs reset a mod-12 ripple counter?

UNIT 3 › SHIFT REGISTERS

B2 · Shift registers

> Anna Univ. EC6302 · Apr/May 2015 · Q13(b)(ii) · 8 marks (also Nov/Dec 2021, 6 marks)
> Explain the functioning of a recirculating shift register with various modes of operation.

✎ **Guess first, on paper: what happens to the data that falls off the end?**



Structure. A row of D flip-flops on one shared clock, with each Q feeding the next D. Every clock edge moves the stored word one place.

Mode	Data in	Data out	Use
SISO	serial	serial	delay line: n clocks of delay
SIPO	serial	all Q at once	serial → parallel (receiver)
PISO	parallel load	serial	parallel → serial (transmitter)
PIPO	parallel load	all Q at once	storage register

SIPO, loading 1011 (MSB first), starting from 0000:

clock	serial in	Q0 Q1 Q2 Q3
0	–	0 0 0 0
1	1	1 0 0 0
2	0	0 1 0 0
3	1	1 0 1 0
4	1	1 1 0 1

After 4 clocks the whole word sits in the register: Q3 holds the first bit, Q0 the last.

Universal shift register (74194). A 4:1 MUX in front of each D chooses the mode with S_1S_0: 00 hold, 01 shift right, 10 shift left, 11 parallel load.

Answer. A **recirculating** shift register feeds the serial output back to the serial input, so data that leaves one end re-enters the other. Nothing is lost, and the word comes back to its starting position after n clocks. The modes to describe are SISO, SIPO, PISO and PIPO, shifting left or right. Recirculating one 1 gives a **ring counter** (B3).

Check B2. A 4-bit SISO register holds 0000. Bits 1, 1, 0, 1 enter on clocks 1–4. On which clock does the first 1 appear at the serial output?

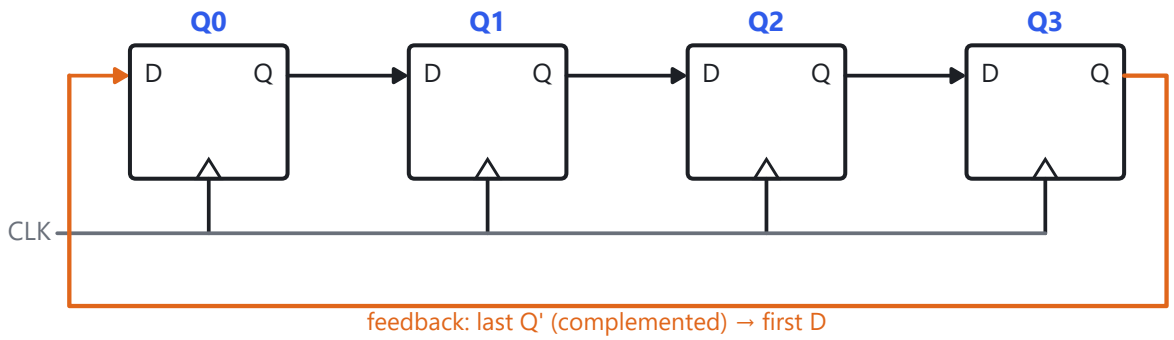
UNIT 3 › SHIFT-REGISTER COUNTERS

B3 · Ring and Johnson counters

Anna Univ. EC6302 · Apr/May 2015 · Q13(b)(i) · 8 marks (Nov/Dec 2021: 7 marks)

Discuss the working of a 4 bit Johnson counter with neat block diagram.

✎ **Guess first, on paper: how many states, and why that many?**

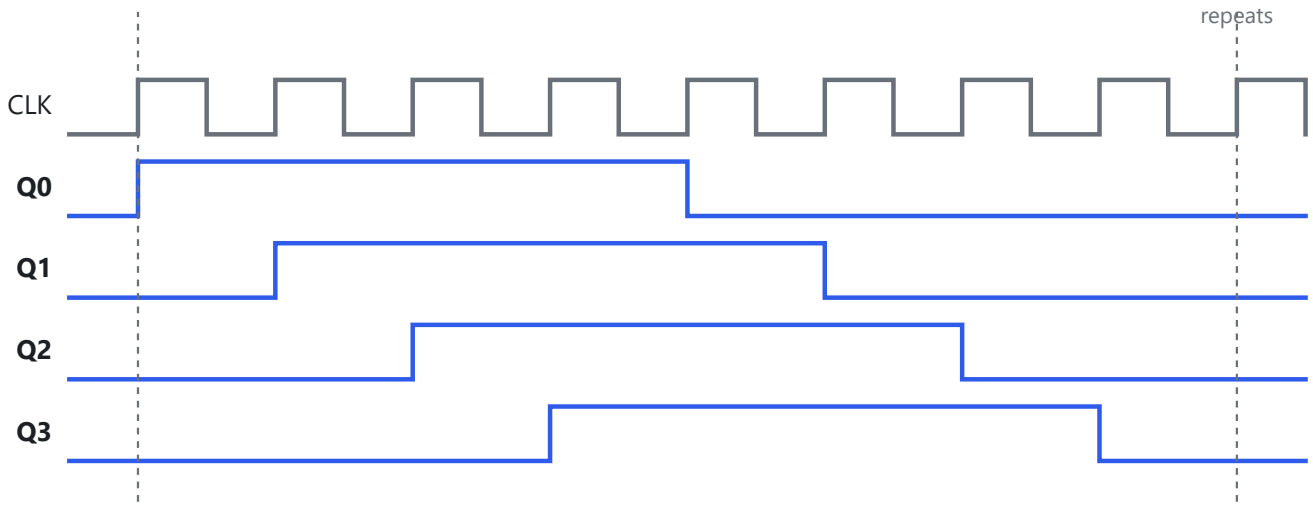


clk	Q0 Q1 Q2 Q3	decode
0	0 0 0 0	Q0’Q3’
1	1 0 0 0	Q0Q1’
2	1 1 0 0	Q1Q2’
3	1 1 1 0	Q2Q3’
4	1 1 1 1	Q0Q3
5	0 1 1 1	Q0’Q1
6	0 0 1 1	Q1’Q2
7	0 0 0 1	Q2’Q3

Ring counter: feed Q3 back to D0. Preset it to 1000 and the single 1 circulates: 1000 → 0100 → 0010 → 0001. n FFs give **mod-n**. Each Q already marks one state, so no decoder is needed.

Johnson (twisted ring): feed **Q3’** back. The register fills with 1s and then empties: n FFs give **mod-2n**. Each state is decoded by one 2-input AND (left table).

Lock-out. 4 FFs have 16 states. The Johnson counter uses 8; the other 8 (e.g. 0100) form a separate loop. A real design adds self-correcting logic or a reset.



Answer outline (8 marks). Block diagram (above) → state table (8 states) → waveforms (each Q is a square wave at $f_{clk}/8$, each one shifted a clock later) → decoding with 2-input ANDs → mod-2n versus ring mod-n → lock-out.

Check B3. (a) How many FFs does a mod-10 Johnson counter need? A mod-10 ring counter? (b) Give the decode term for state 0011 of the 4-bit Johnson counter.

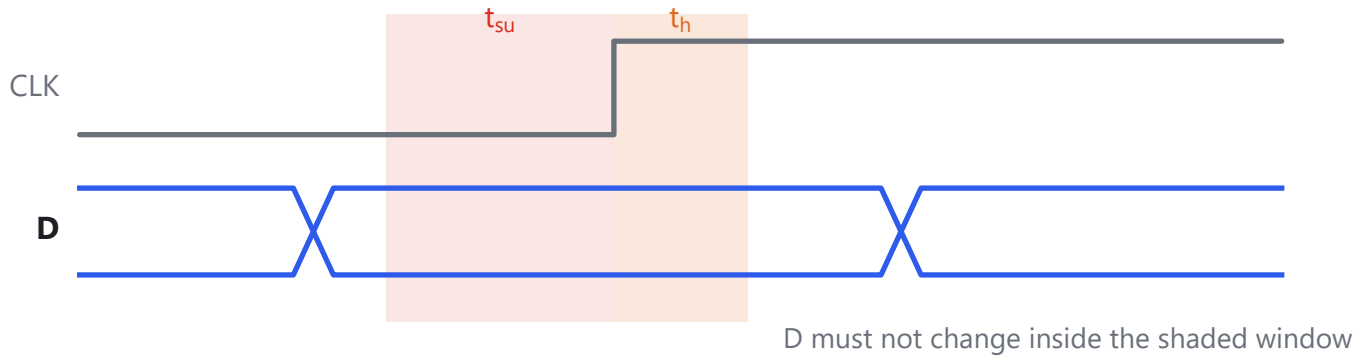
UNIT 3 › TIMING ANALYSIS

B4 · Timing analysis of sequential circuits

practice · no real paper item found · 4 marks

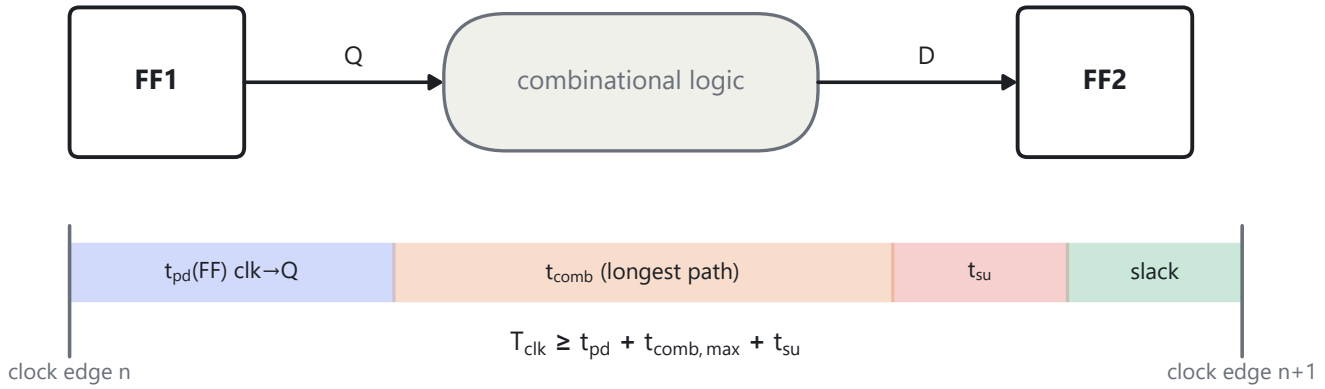
A synchronous circuit has $t_{pd,FF}$ = 8 ns, t_{su} = 3 ns, t_h = 2 ns, and combinational delays of 12 ns on the longest path and 1 ns on the shortest. Find f_{max} and check the hold time.

✎ **Guess first, on paper.**



D must not change inside the shaded window

Three numbers per flip-flop. $\mathbf{t_{su}}$ (setup): D must be stable *before* the edge. $\mathbf{t_h}$ (hold): D must stay stable *after* the edge. $\mathbf{t_{pd}}$: time from the edge until Q is valid. Break setup or hold and the FF can go **metastable**: its output sits between 0 and 1 for an unpredictable time.



Setup (longest path): data launched at edge n must reach FF2 a setup time before edge n+1, so $T_{clk} \geq t_{pd} + t_{comb,max} + t_{su}$, which gives $f_{max} = 1 / (\text{that sum})$.
Hold (shortest path): new data must not arrive while FF2 is still holding the old value, so $t_{pd} + t_{comb,min} \geq t_h$. The clock period is not in this inequality, so **a slower clock cannot fix a hold violation**.

> **Answer.** $T_{min} = 8 + 12 + 3 = 23$ ns, so $f_{max} = 1/23$ ns ≈ **43.5 MHz**. Hold: 8 + 1 = 9 ns ≥ 2 ns, so **hold is met**.

Check B4. The same circuit gains 4 ns of extra logic on its longest path. Find the new f_{max}. Does the hold result change?

Part B: answers to the Checks

1. **B1** (a) 4 × 25 = 100 ns. (b) Mod-12 → clear at 12 = 1100, so NAND(Q_3, Q_2) → CLR′.
2. **B2** The first bit in is a 1, and it reaches Q3 on clock 4. It appears at the serial output then (4 clocks of delay).
3. **B3** (a) Johnson needs 2n = 10, so 5 FFs; ring needs n = 10, so 10 FFs. (b) 0011 → Q1′Q2.
4. **B4** T = 8 + 16 + 3 = 27 ns, so f_{max} ≈ 37.0 MHz. The hold result does not change: it depends on the shortest path, which did not change.

PART C

State machines (Unit 4)



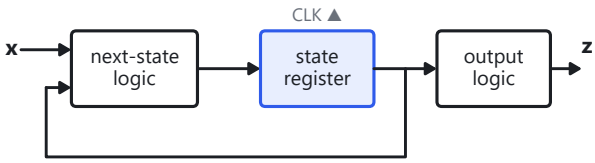
UNIT 4 › MODEL

C1 · Every synchronous circuit is one structure

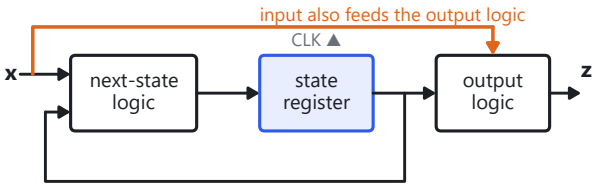
Anna Univ. EC6302 · Nov/Dec 2021 · Part A Q5 · 2 marks

What are the classifications of sequential circuits?

✎ **Guess first, on paper.**



Moore: z = f(state)



Mealy: z = f(state, x)

A **finite-state machine** has three parts: a state register (the flip-flops), next-state logic, and output logic. The only difference between the two models is **where the output comes from**.

	Moore	Mealy
Output depends on	state only	state **and** present input
Output changes	only after a clock edge, stable all cycle	whenever the input changes, so it can glitch
States needed	often one more	fewer
Response	one clock later	in the same clock cycle

Answer. By timing: **synchronous** (changes only at clock edges) and **asynchronous** (changes as soon as inputs change; no clock). By output: **Moore** (z = f(state)) and **Mealy** (z = f(state, input)).

UNIT 4 › DESIGN

C2 · Designing a synchronous FSM: the 010 detector

Anna Univ. EC6302 · Apr/May 2015 · Q15(b)(ii) · 14 marks

Design a sequential pattern detector that receives a stream of input bits. The circuit should recognize the pattern 010 and produce an output whenever this pattern is received.

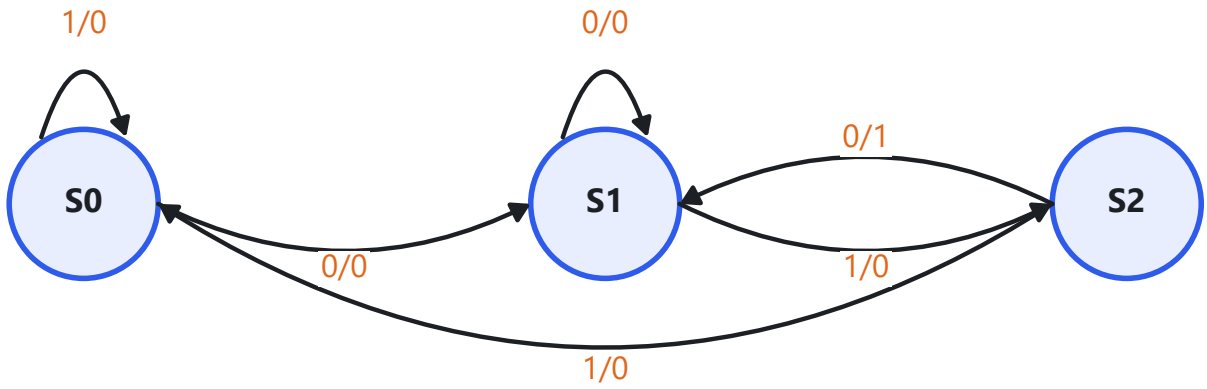
✎ **Guess first, on paper: what should each state *remember*?**

The procedure (also asked on its own: Anna Univ. Nov/Dec 2021 Q15(a)(i), 10 marks, "Summarize the design procedure"):

1. Word spec → **state diagram**. Each state stands for "how much of the pattern I have seen so far".
2. **State table**: present state × input → next state and output.

3. **Reduce** the states (C3).
4. **Assign** binary codes. $\lceil \log_2(\text{number of states}) \rceil$ FFs.
5. Choose an FF type and build the **excitation table**.
6. **K-map** each FF input and each output, using unused codes as don't-cares.
7. Draw the circuit and check that unused states lead back into the machine.

Mealy version. S0 = nothing useful seen, S1 = seen "0", S2 = seen "01". Label = input/output. Overlap is allowed: after 010 the last 0 already starts a new match, so S2 goes to S1 on input 0.



PS (AB)	x=0: NS, z	x=1: NS, z
S0 = 00	01, 0	00, 0
S1 = 01	01, 0	10, 0
S2 = 10	01, 1	00, 0
11 unused	X	X

With D FFs, D = next state, so read the K-maps straight off the table (11 = X):

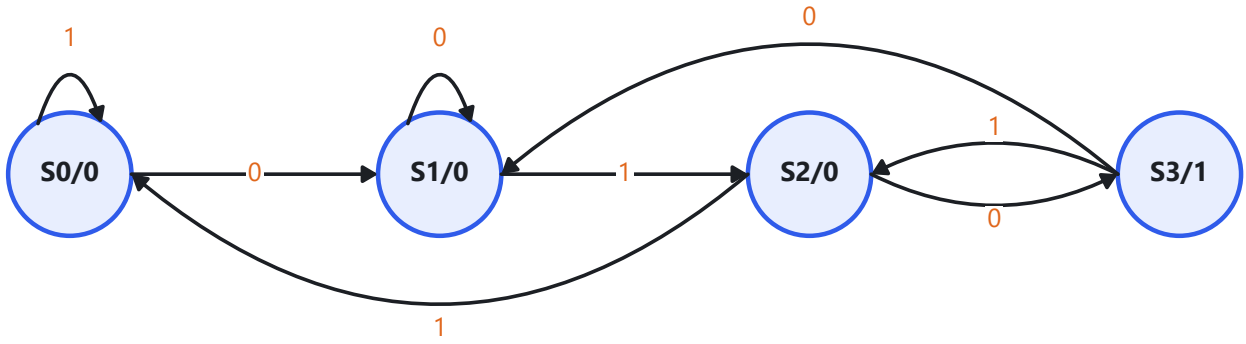
$D_A = Bx$

$D_B = x'$

$z = Ax'$

Simulated on x = 0101001011010 → z = 0010100100001 (a 1 each time 010 completes).

Moore version. The output must belong to a state, so add S3 = "seen 010" with z = 1. From S3: input 0 → S1, input 1 → S2 (the "01" overlap).



$D_A = Bx + AB'x' \cdot D_B = x' \cdot z = AB$ (S0=00, S1=01, S2=10, S3=11)

Answer outline (14 marks). State diagram → state table → assignment → excitation (D) → K-maps → equations → circuit (2 D-FFs plus the gates above). Mealy needs 3 states and z appears in the same clock as the final 0. Moore needs 4 states and z appears one clock later.

Check C2. (a) Draw the Mealy state diagram for **non-overlapping** 010 detection. Which single arrow changes? (b) How many states does a Moore detector for 1011 (overlapping) need?

UNIT 4 › STATE REDUCTION

C3 · State reduction

practice · constructed table · no real paper item found · 4 marks

Reduce the state table below to its minimum number of states.

✎ **Guess first, on paper: which rows look like twins?**

PS	NS x=0	NS x=1	z x=0	z x=1
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	g	f	0	1
g	a	f	0	1

Rule. Two states are **equivalent** if, for every input, they give the same output *and* go to equivalent next states. Merging them never changes how the machine behaves.

Partition method.
P_0 (split by outputs): (abc)(defg)
P_1: inside (abc), on x=1 a → b but b, c → d, so (a)(bc). Inside (defg), on x=0 d → e and f → g, but e, g → a, so (df)(eg).
P_2: on x=0, b → c ∈ (bc) but c → a ∈ (a), so (a)(b)(c)(df)(eg).
$P_3 = P_2$ → stop.

Answer. d ≡ f and e ≡ g, so **7 → 5 states**. Replace f with d and g with e everywhere. Here both still need $\lceil \log_2 \rceil = 3$ FFs, but the next-state logic gets more don't-cares. Reduction does not always save a flip-flop.
(Implication chart = the same test done pairwise on a triangle of cells.)

Check C3. Why must the outputs match first, before you compare next states?

UNIT 4 › TIMING ISSUES

C4 · Timing issues in synchronous circuits

practice · no real paper item found · 4 marks

A push-button signal enters a 100 MHz synchronous system directly. What can go wrong, and how is it fixed?

✏️ **Guess first, on paper.**

- **Clock skew.** The clock reaches different FFs at different times. If skew works against the data direction it eats into the **hold** margin, and a slower clock cannot fix that. If it works with the data direction it eats into setup. Fix it with careful clock-tree layout.
- **Output glitches.** Mealy outputs, or any decoded state, can glitch while the state bits change. If a glitch matters, register the output (use Moore style).
- **Asynchronous inputs → metastability.** An input that is not timed to the clock will sometimes violate t_{su}/t_h. Fix: a **2-FF synchronizer** (two D-FFs in series on the system clock). The second FF gives the first a full clock period to settle. This makes failure exponentially rarer but never impossible.

Answer. The button can change inside a setup/hold window, so the FF may go metastable and different FFs may read different values. Fix: debounce the switch (a monostable or an RC with a Schmitt trigger, C6), then pass it through a 2-FF synchronizer before any logic uses it.

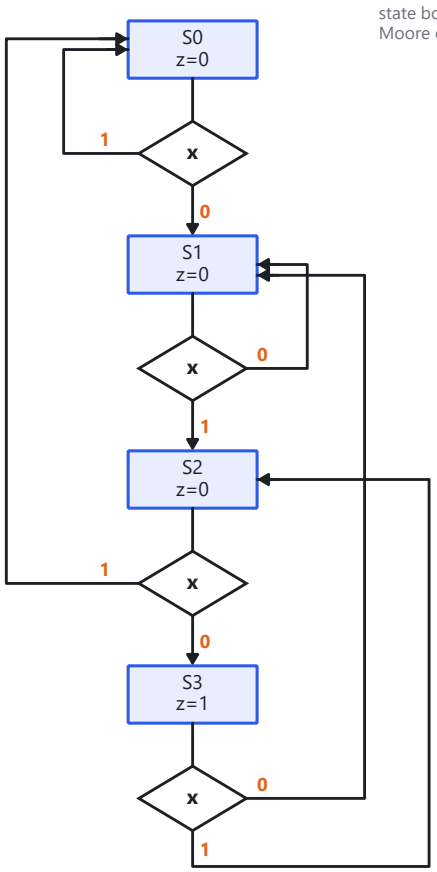
UNIT 4 › ASM

C5 · Algorithmic State Machine (ASM) charts

Anna Univ. EC6302 · Apr/May 2015 · Q15(a)(i) · 8 marks

Explain how a state graph for a sequential machine can be converted to an equivalent ASM chart.

✏️ **Guess first, on paper: what in a flowchart could stand for "one clock"?**



Three symbols.

- **State box** (rectangle): one state, which lasts one clock. Moore outputs are written inside it.
- **Decision box** (diamond): tests an input; 1 and 0 exits.
- **Conditional output box** (oval): a Mealy output that is active only on that path.

An **ASM block** is one state box plus all the decisions and conditional outputs reachable from it in the same clock.

Conversion rule. Each state-graph node becomes a state box. Its outgoing arcs become a decision tree on the inputs. Each Moore output goes inside the state box. Each Mealy output (on an arc) becomes an oval on that branch.

Left: the Moore 010 detector from C2 as an ASM chart.

Answer outline (8 marks). The three symbols → definition of an ASM block → the four-step conversion rule → one converted example (the left figure). Advantage: it reads like an algorithm and maps directly onto the next-state and output logic.

Check C5. In the Mealy 010 detector, z = 1 on the arc S2 —0→ S1. Where does z go in the ASM chart?

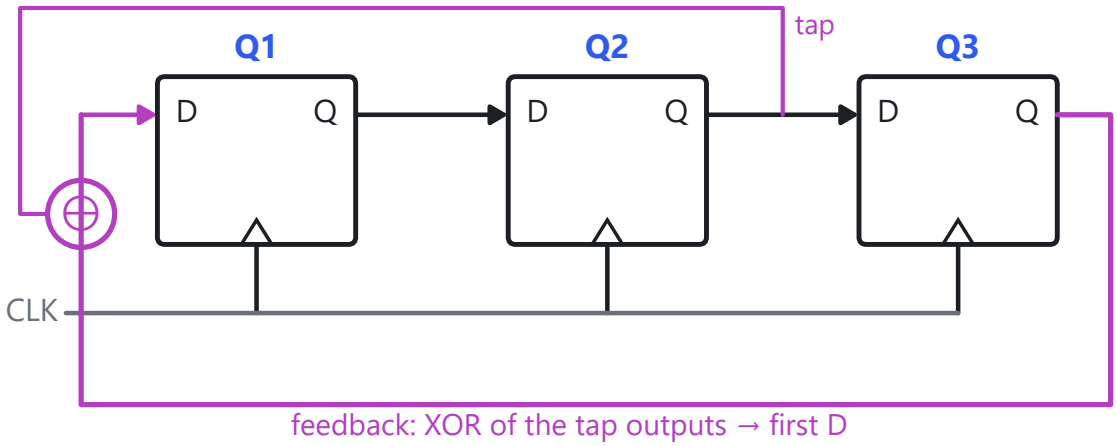
UNIT 4 › SYNCHRONOUS APPLICATIONS

C6 · Pulse trains, pseudo-random sequences, clock generation

practice · no real paper item found · 4 marks

A 3-bit LFSR has feedback polynomial $x^3 + x + 1$ ($D_1 = Q_2 \oplus Q_3$) and starts at $Q_1Q_2Q_3 = 100$. List the states and find the period.

✎ **Guess first, on paper.**



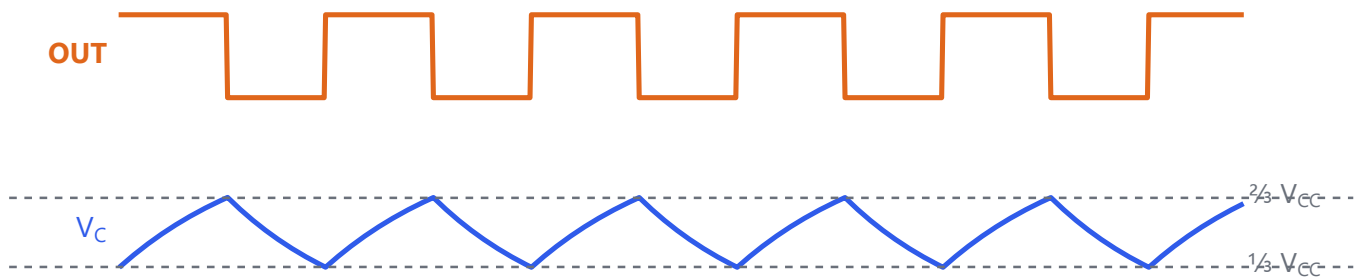
PRBS generator = LFSR. A shift register whose input is the XOR of chosen "tap" outputs. With a **primitive** polynomial it runs through all $\mathbf{2^n - 1}$ non-zero states before repeating (a maximal-length or m-sequence). **All-zeros is a lock-out**: $0 \oplus 0 = 0$ forever, so the seed must be non-zero. The output looks random but repeats exactly, which makes it useful for test patterns, scramblers and spread spectrum.

> **Answer.** 100 → 010 → 101 → 110 → 111 → 011 → 001 → 100. **Period 7 = $\mathbf{2^3 - 1}$**, so this LFSR is maximal-length. The serial output (Q_3) is 0 0 1 0 1 1 1, repeating.

Pulse-train generator. Emits a fixed bit pattern, one bit per clock, over and over. Two easy builds: a recirculating shift register preloaded with the pattern (B2), or a mod-N counter followed by decode logic. The period equals the number of states.

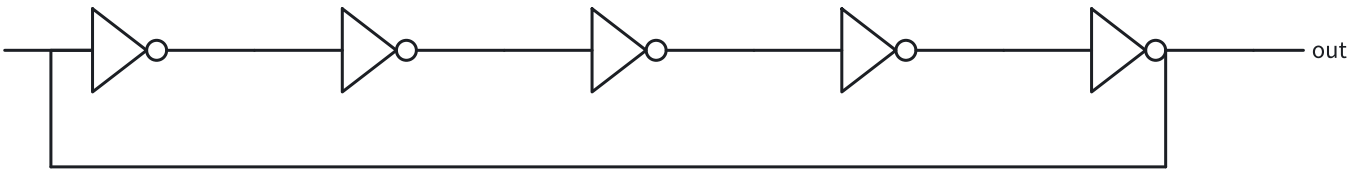
Clock generation. Dividing an existing clock: a mod-N counter gives f/N, and k toggle FFs give $f/2^k$. Making the clock in the first place is the job of a **multivibrator**:

Type	Stable states	Gives
Astable	0	free-running square wave, i.e. a clock
Monostable	1	one fixed-width pulse per trigger (debounce, timing)
Bistable	2	a latch / flip-flop



rise: charge through R_1+R_2, $t_H = 0.693(R_1+R_2)C$ fall: discharge through R_2, $t_L = 0.693 \cdot R_2 \cdot C$

555 astable. C charges through R_1+R_2 up to $\frac{2}{3}V_{CC}$, then discharges through R_2 down to $\frac{1}{3}V_{CC}$. $f = 1.44 / ((R_1 + 2R_2)C)$. Duty cycle $= (R_1+R_2)/(R_1+2R_2)$, always > 50% in this circuit. Example: R_1 = 1 kΩ, R_2 = 10 kΩ, C = 10 nF gives f ≈ 6.86 kHz and duty ≈ 52%. **555 monostable:** pulse width $T = 1.1RC$.



Ring oscillator. An **odd** number N of inverters in a loop has no stable state, so it oscillates at $f = 1/(2N \cdot t_{pd})$. Example: 5 inverters with 2 ns each give 50 MHz. It drifts with temperature and supply voltage. A **crystal oscillator** gives the precise system clock. A **Schmitt trigger** (two thresholds, with hysteresis) turns slow or noisy edges into clean ones.

Check C6. (a) What is the period of a maximal-length 5-bit LFSR? (b) Why must a ring oscillator have an odd number of inverters?

UNIT 4 › ASYNCHRONOUS CIRCUITS

C7 · Asynchronous circuits: races and hazards

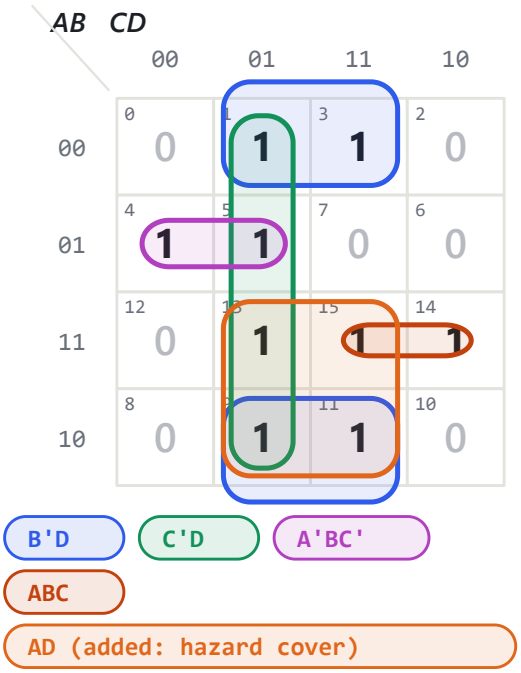
> Anna Univ. EC6302 · Nov/Dec 2021 · Q15(b) · 13 marks
>
> What is the objective of state assignment in a asynchronous circuit? Give the hazard free realization for the Boolean function f (A, B, C, D) = ΠM (0, 2, 6, 7, 8, 10, 12).

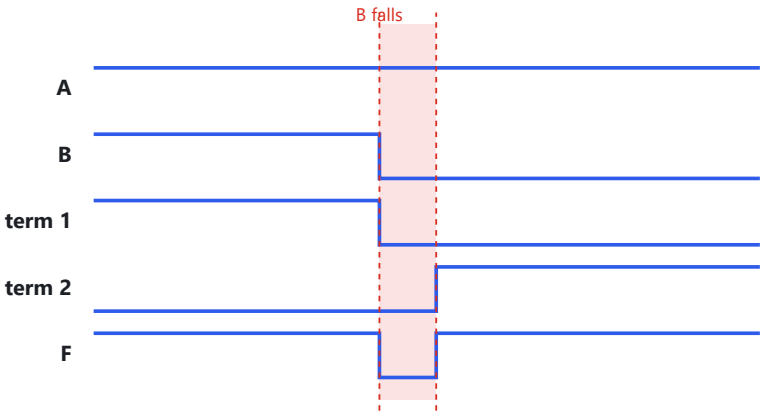
✎ **Guess first, on paper: what goes wrong when two state bits must change "at once" with no clock?**

Asynchronous = no clock. Outputs and state react directly to input changes, through feedback delays. Work in **fundamental mode**: change only one input at a time, and only after the circuit has settled. A **stable state** is one where next state = present state.

Races. If one transition needs two state variables to change, the delays decide which changes first. In a **non-critical** race every order ends in the same stable state. In a **critical** race the final state depends on the order, which is a bug. **The objective of state assignment** is to give codes that differ in **only one bit** to states that transition into each other, so critical races cannot happen.

Hazards. A **static-1 hazard** is a momentary 0 when F should stay at 1, because one product term turns off before the next one turns on. Static-0 is the dual, dynamic hazards bounce several times, and essential hazards are built into the flow table.





static-1 hazard: term 1 turns off before term 2 turns on, so F dips

The cure. On the K-map, every pair of **adjacent 1s** must sit inside **one common group**. Add the **consensus** (redundant) group for any adjacent pair that crosses between two groups.

Answer. ΠM(0,2,6,7,8,10,12) ⇒ the 1s are at Σm(1,3,4,5,9,11,13,14,15). Minimal cover: f = B′D + C′D + A′BC′ + ABC. Two adjacent pairs cross between groups: m11–m15 (B changes) and m13–m15 (C changes). One extra prime implicant, AD, covers both.
Hazard-free: f = B′D + C′D + A′BC′ + ABC + AD.

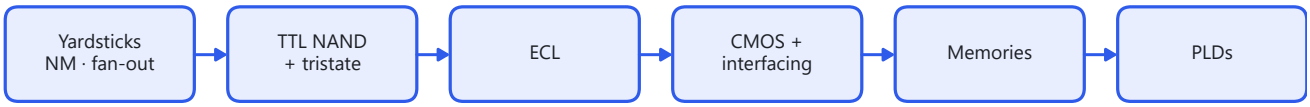
Check C7. F = AB + A′C. Which input change can glitch, and which term removes the glitch?

Part C: answers to the Checks

1. **C2** (a) Non-overlapping: after a match, S2 —0/1→ **S0** instead of S1, because the last 0 cannot start the next match. (b) 1011 Moore needs 5 states (0, 1, 10, 101, 1011 with z = 1).
2. **C3** If two states give different outputs for some input, an observer can already tell them apart, so they can never be merged. Next states only matter among states whose outputs agree.
3. **C5** In an oval (conditional output box) on the x = 0 exit of S2's decision box. It is not in a state box, because z depends on the input.
4. **C6** (a) $2^5 - 1 = 31$. (b) With an even number of inverters the loop has a stable state (like a latch), so it never oscillates.
5. **C7** With B = C = 1, A changing from 1 to 0 hands over from AB to A′C and can glitch. Add the consensus term **BC**: F = AB + A′C + BC.

PART D

Logic families, memories, PLDs (Unit 5)



UNIT 5 › YARDSTICKS

D1 · Noise margin, fan-out, delay, power

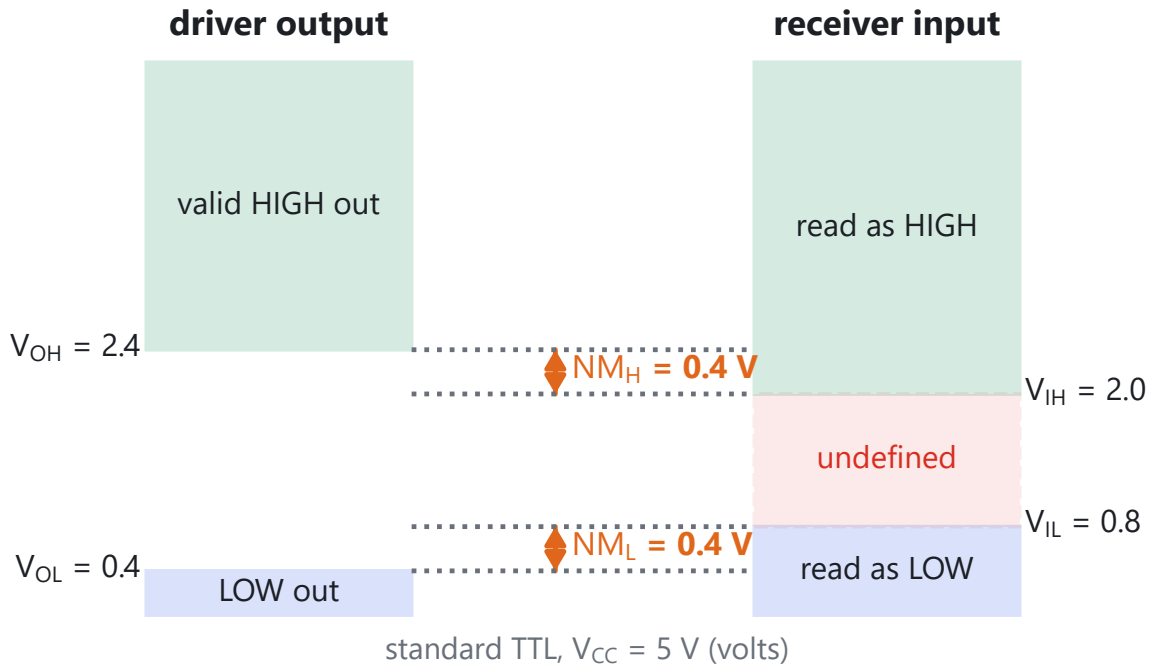
Anna Univ. EC6302 · Nov/Dec 2021 · Part A Q2 · 2 marks

Define Noise margin.

RTU (3E2075, 2012) Digital Electronics · Unit II Q(a) · 8 marks

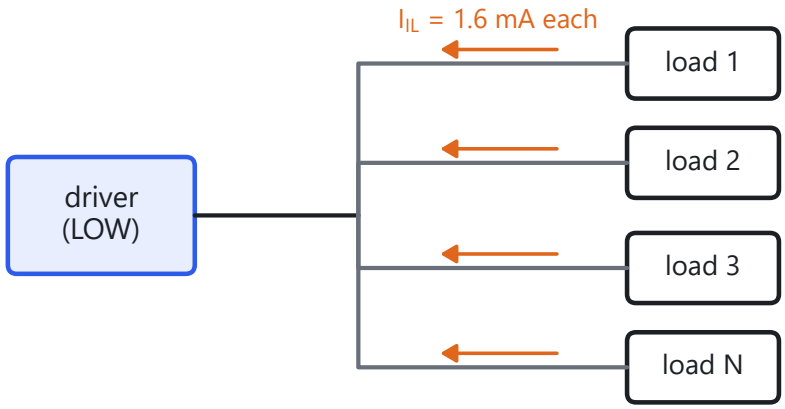
Define the (i) noise immunity and (ii) Fanout.

✎ **Guess first, on paper. Then: why must "1" be a *range* of voltages?**



standard TTL, V_{CC} = 5 V (volts)

Noise margin is how much noise voltage a signal can pick up between one gate's output and the next gate's input and still be read correctly: $NM_H = V_{OH(min)} - V_{IH(min)}$, $NM_L = V_{IL(max)} - V_{OL(max)}$. The guaranteed output is always better than what the input needs, and that gap is the margin.



LOW output must SINK $N \times I_{IL} \le I_{OL} = 16$ mA → $N \le 10$

Fan-out is the number of gate inputs one output can drive while staying in spec. It is limited by current in both states: fan-out = min(I_{OL}/I_{IL}, I_{OH}/I_{IH}). **Fan-in** is the number of inputs a gate has.

Standard TTL (74xx), V_{CC} = 5 V			
$V_{OH(min)}$ 2.4 V	$V_{OL(max)}$ 0.4 V	$V_{IH(min)}$ 2.0 V	$V_{IL(max)}$ 0.8 V
I_{OH} 400 µA	I_{OL} 16 mA	I_{IH} 40 µA	I_{IL} 1.6 mA

⇒ $NM_H = NM_L$ = **0.4 V**; fan-out = min(16/1.6, 400/40) = **10**. t_{pd} ≈ 10 ns, P ≈ 10 mW, so the **power–delay product** ≈ 100 pJ. That figure of merit is the energy per switching event.

> **Answer. Noise margin** (noise immunity): the largest noise voltage that can be added to a valid logic level without changing how it is read. Give both formulas above. **Fan-out**: the maximum number of standard loads one output can drive without its levels leaving spec. It is set by the current ratios.

Check D1. A family has I_{OL} = 8 mA, I_{IL} = 0.4 mA, I_{OH} = 400 µA, I_{IH} = 20 µA. Find the fan-out.

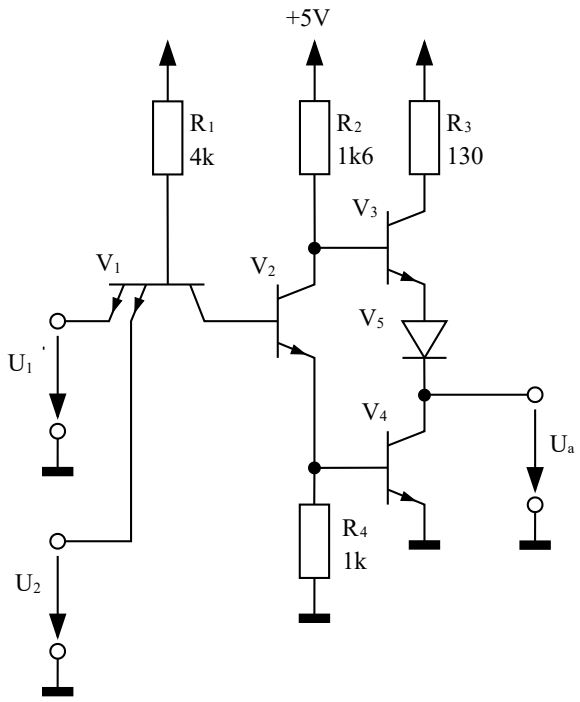
UNIT 5 › TTL

D2 · The TTL NAND gate (totem pole)

practice · textbook-style (Jain, TTL chapter) · 8 marks

Draw the circuit of a standard TTL NAND gate with totem-pole output and explain its operation. Why can't two totem-pole outputs be tied together?

✎ **Guess first, on paper.**



Standard 7400 TTL NAND, totem-pole output · Wikimedia Commons, CC BY 2.0 de. German labels: V_1…V_4 = transistors Q1…Q4, V_5 = diode D, U = voltage.

- **Q1 (multi-emitter)** does the AND. Each emitter is one input.
- **Any input LOW** → Q1's base current flows out through that emitter → Q2 OFF → Q4 OFF, and Q3 gets base current through R_2 → Q3 + D pull the output **HIGH**.
- **All inputs HIGH** → Q1's base–collector junction drives Q2 ON → Q4 saturates, output **LOW**. Q2's collector drops, and the diode D makes sure Q3 stays OFF.
- **Totem pole** (Q3 on top of Q4): there is an active transistor both pulling up and pulling down, so output impedance is low in both states and edges are fast.
- **Never tie two outputs together.** If one drives HIGH while the other drives LOW, Q3 of one gate shorts to Q4 of the other through only 130 Ω, and a large current flows. For shared lines use **open-collector** (external pull-up, gives wired-AND) or **tristate**.
- An unconnected TTL input acts as a HIGH (no emitter current flows), but it picks up noise. Tie unused inputs to a defined level.

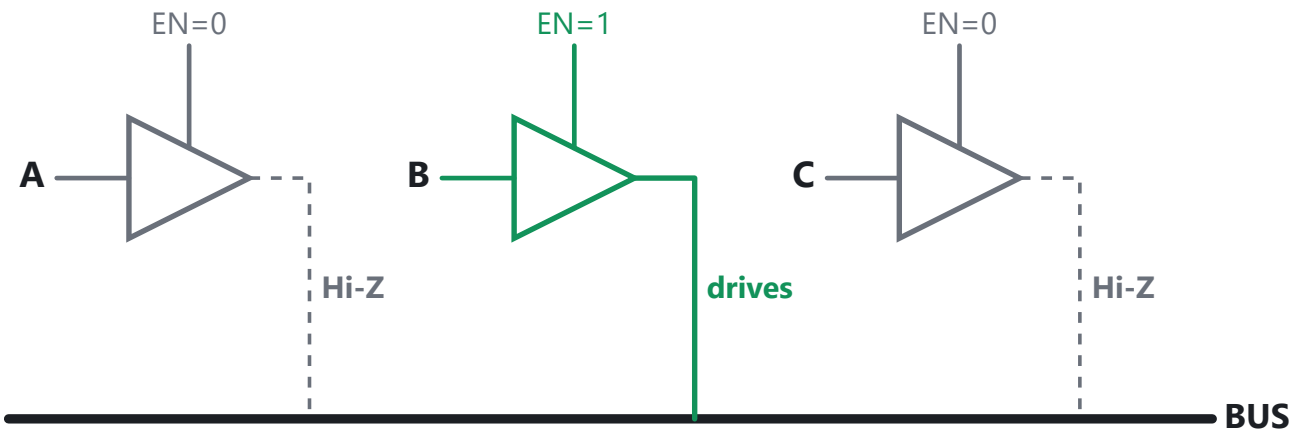
Sub-families trade speed against power: 74 (standard), 74L (low power), 74H (high speed), 74S (Schottky: a clamp stops saturation, so it is faster), **74LS** (low-power Schottky, the workhorse), 74AS/ALS/F.

UNIT 5 › TRISTATE

D3 · Tristate TTL

> Anna Univ. EC6302 · Nov/Dec 2021 · Q11(b)(i) · 8 marks
>
> Draw and explain Tri-state TTL inverter circuit diagram with its operation.
>
> Anna Univ. EC1201 · Nov/Dec 2021 · Part A Q3 · 2 marks
>
> What is tristate logic? What are the demerits?

✎ **Guess first, on paper: how can one output be "neither 0 nor 1"?**



exactly one enable is 1 at a time; disabled outputs float (high impedance)

Third state = high impedance (Hi-Z). When the enable input is inactive, the output is electrically disconnected. This lets many outputs share one **bus**, with only the enabled one driving it.

Circuit. Start from the totem-pole gate above and add an enable input E. It connects (1) to an **extra emitter of Q1** and (2) through a **diode to Q2's collector** (Q3's base). When disabled (E LOW), the extra emitter steals Q1's base current, so Q2 and Q4 are OFF, and the diode pulls Q3's base LOW, so Q3 is OFF too. **Both totem-pole transistors are OFF, so the output is Hi-Z.** When enabled (E HIGH), the diode is reverse-biased and the gate works normally.

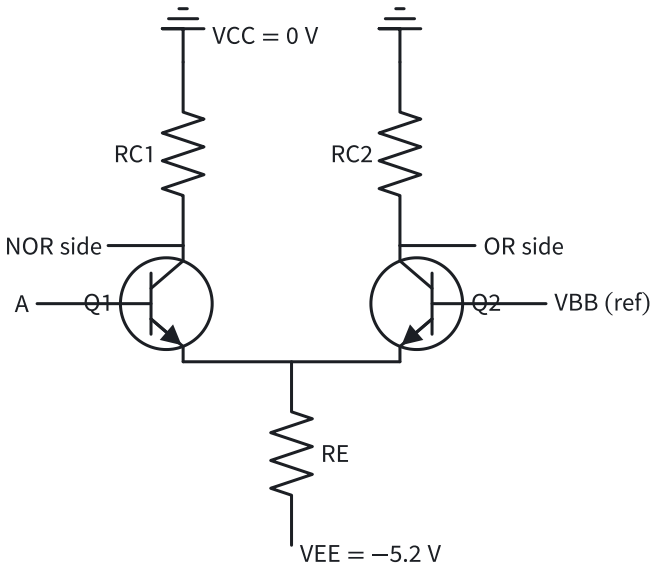
> **Answer.** Tristate logic has outputs 0, 1 and Hi-Z, selected by the enable. Merit: it keeps the fast active pull-up (unlike open-collector) and still allows bus sharing. **Demerits:** if two enables are on at once, two outputs fight (bus contention, large current), so control logic must arbitrate. A bus with every output in Hi-Z floats, so it needs pull-ups or bus-keepers. Enable/disable times add delay.

UNIT 5 › ECL

D4 · Emitter-coupled logic

> RTU (3E2075) · Unit II Q(b) · 8 marks
>
> Explain the parameters used to characterize logic families. Calculate the noise margin of ECL gate.
>
> Anna Univ. EC1201 · Nov/Dec 2021 · Q12(b)(i) · 10 marks
>
> Compare the features of TTL, ECL and HTL families.

✎ **Guess first, on paper: what makes a bipolar transistor slow to turn OFF?**



A current switch. R_E sets a nearly constant current. Whichever transistor, Q1 (input A) or Q2 (fixed reference V_{BB}), has the higher base takes that current. The current is **steered**, never switched off, and neither transistor **saturates**.

Why it is the fastest family (about 1 ns). A saturated BJT stores extra charge that must be removed before it can turn off. ECL never saturates, so it never pays that delay.

Costs. Highest power (the current always flows). Small voltage swing (logic 1 ≈ −0.8 V, logic 0 ≈ −1.7 V, about 0.8 V apart), so the noise margin is small. It runs from a negative supply (V_{EE} = −5.2 V, V_{CC} = 0). It gives both OR and NOR outputs.

Noise margin, method. Same formulas as D1, applied to ECL levels. With typical 10K-series values $V_{OH(min)}$ = −0.98 V, $V_{IH(min)}$ = −1.105 V, $V_{IL(max)}$ = −1.475 V, $V_{OL(max)}$ = −1.63 V: NM_H = −0.98 − (−1.105) = **0.125 V**, NM_L = −1.475 − (−1.63) = **0.155 V**. These levels are illustrative. Datasheet values vary with series and temperature; use the levels the paper gives you.

	TTL (74/74LS)	ECL	CMOS (74HC)
Device	BJT, saturating	BJT, non-saturating	MOSFET, complementary
Speed	~10 ns	**fastest**, ~1 ns	~10 ns (newer series faster)
Static power	mW per gate	**highest**	**≈ 0**
Noise margin	0.4 V	smallest, ~0.1–0.15 V	largest, ~0.3–0.45 V_{DD}
Fan-out	10	high	very high (limited by capacitance)
Supply	+5 V	−5.2 V	3–15 V (4000), 2–6 V (HC)

HTL (high-threshold logic, named in the EC1201 question) is an older bipolar family for noisy industrial settings: a high supply (~15 V) and large noise margin, but slow. It does not appear in our syllabus text.

UNIT 5 › CMOS

D5 · CMOS gates and interfacing

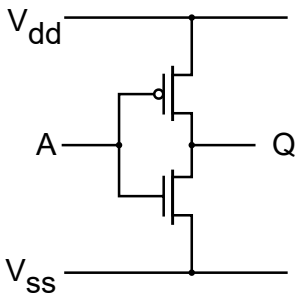
Anna Univ. EC1201 · Nov/Dec 2021 · Q12(a)(i) · 10 marks

Draw a CMOS NAND gate and explain its operation. List the characteristics of CMOS Logic family.

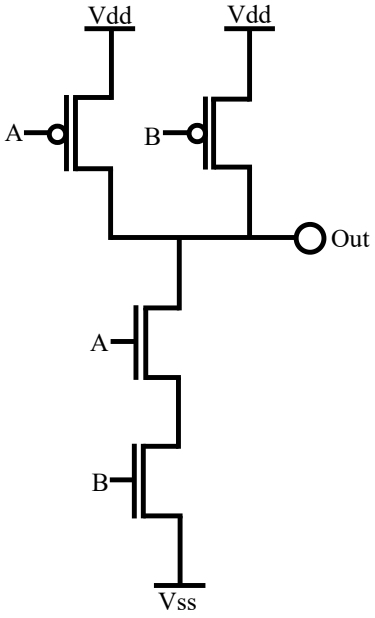
RTU (3E2075) · Unit II OR-Q(a) · 8 marks

What are the different schemes for CMOS to TTL interface. Explain one of them.

✎ **Guess first, on paper: when is there *no* path from V_{DD} to ground?**



CMOS inverter · Wikimedia Commons, public domain



CMOS NAND · Wikimedia Commons, CC BY-SA 3.0

Rule. Each gate has a PMOS **pull-up network** and an NMOS **pull-down network** that are logical complements, so exactly one of them conducts. **NAND:** NMOS in **series** (both inputs must be 1 to pull the output LOW) and PMOS in **parallel** (any 0 pulls it HIGH). NOR is the reverse.

A B	NMOS (series)	PMOS (parallel)	Out
`0 0`	`both off`	`both on`	`1`
`0 1 / 1 0`	`one off → open`	`one on`	`1`
`1 1`	`both on`	`both off`	`0`

Characteristics. Static power ≈ 0, because in steady state no path conducts. **Dynamic power $P = \alpha C V_{DD}^2 f$**, so power rises with frequency. Output swings rail to rail. Noise margin is large. Fan-out is very high (the input is a gate capacitor). Supply range is wide, density is high, and it scales well, which is why CMOS took over. Weak points: it is sensitive to static (ESD), and it is slower with large capacitive loads.

Handling (EC1201 Q12(b)(ii), 6 marks). Store parts in conductive foam. Ground yourself and the bench. Never insert or remove a part with power on. **Tie every unused input** to V_{DD} or GND, because a floating CMOS input wanders and can turn both networks partly on.

Interfacing: always compare levels *and* currents.

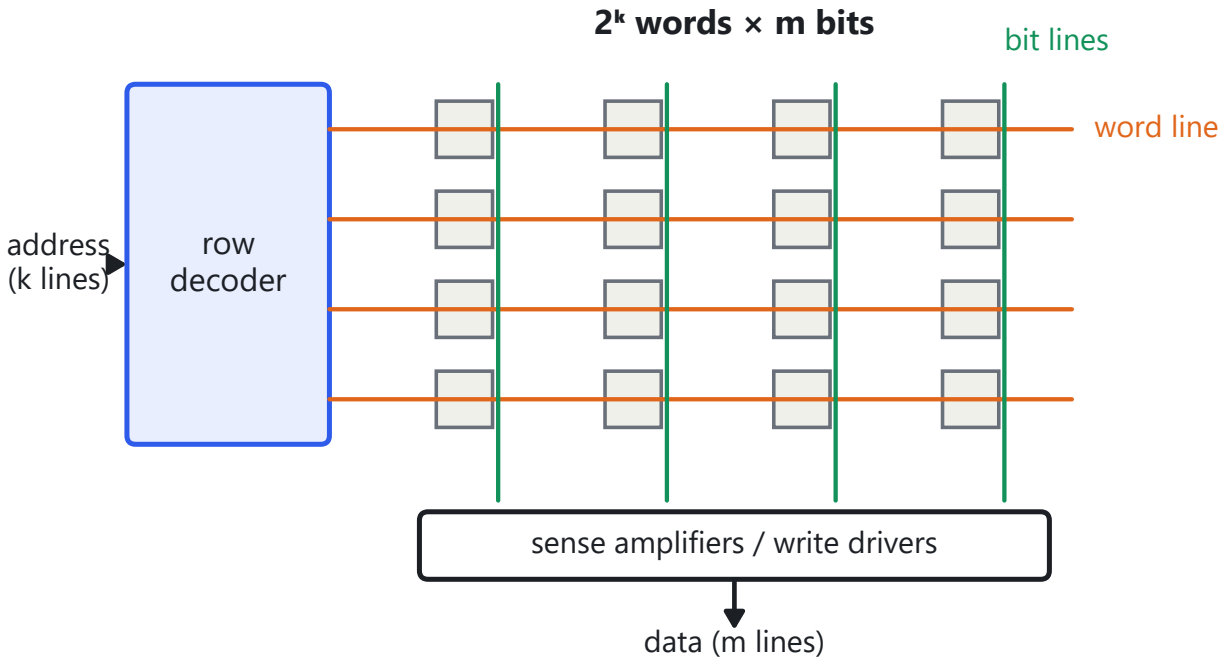
- **TTL → CMOS (5 V):** TTL's $V_{OH(min)}$ = 2.4 V is below a 74HC input's V_{IH} ≈ 3.5 V. Fix: a **pull-up resistor** to +5 V, or use **74HCT** (CMOS with TTL input thresholds).
- **CMOS → TTL:** the voltage levels are fine, but a CMOS output must sink TTL's I_{IL} = 1.6 mA per load. 74HC can sink this for a few loads. 4000-series cannot, so add a **buffer** (e.g. 4049/4050) or drive LS-TTL (lower I_{IL}). With different supply voltages, use a level-shifter.

Check D5. (a) Draw the network for a CMOS NOR: which network is in series? (b) Doubling the clock frequency and halving V_{DD} changes the dynamic power by what factor?

D6 · Semiconductor memories

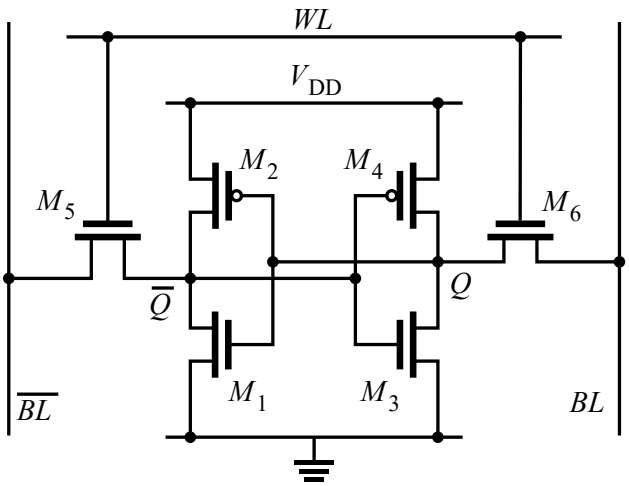
> Anna Univ. EC6302 · Nov/Dec 2021 · Q14(a) · 13 marks
> Write the differences between static and dynamic RAM. Draw the circuits of one cell of each and explain its working.
> Anna Univ. EC6302 · Apr/May 2015 · Q14(a)(i) · 8 marks
> Explain memory decoding. Compare the RAM, ROM, PROM & EPROM.

✎ **Guess first, on paper: which cell needs refreshing, and why?**

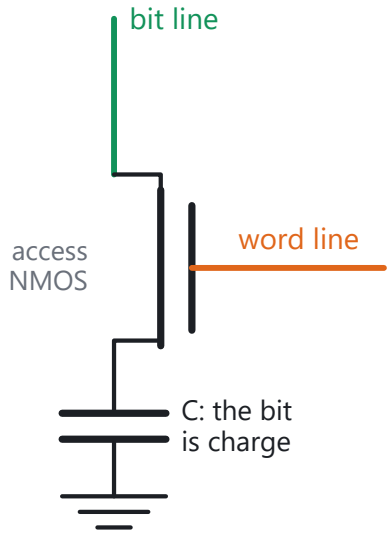


Organisation. 2^k words × m bits: k address lines, m data lines. The row decoder raises one **word line**, and that row's cells connect to the **bit lines**. Example: 1K × 8 → 2^{10} words → **10 address lines, 8 data lines**, 8192 bits.

Memory decoding (building a big memory from small chips). For 4K × 8 from four 1K × 8 chips: A_9–A_0 go to every chip, and $A_{11}A_{10}$ go to a **2:4 decoder** whose outputs drive each chip's **chip-select**. Each chip then occupies its own 1K block of addresses.



6T SRAM cell · Wikimedia Commons, public domain



1T-1C DRAM cell · generated

	SRAM	DRAM
Cell	6 transistors: two cross-coupled inverters (a latch) + 2 access transistors	1 transistor + 1 capacitor
Stores the bit as	the latch state (feedback holds it)	charge on C
Refresh	none (holds while powered)	**needed** every few ms: charge leaks; reads are destructive
Speed / density / cost per bit	fast / low / high	slower / high / low
Use	cache	main memory

Working. SRAM: the word line turns on M5/M6, the latch drives BL and BL′, and a sense amplifier reads the difference. To write, force BL/BL′ hard enough to flip the latch. DRAM: the word line turns on the transistor and C shares its charge with the bit line. That small change is sensed, and the value is then **written back**.

Type	Write	Erase	Volatile?
RAM (SRAM/DRAM)	any time, fast	–	yes
Mask ROM	at manufacture	never	no
PROM	once (blow fuses)	never	no
EPROM	programmer	UV light, whole chip, through a quartz window	no
EEPROM / Flash	in circuit	electrically (byte / block for Flash)	no

Check D6. (a) How many address lines does a 16K × 4 memory need, and how many of its chips make a 64K × 8 memory? (b) EPROM vs EEPROM in one line.

UNIT 5 › PLDS

D7 · Programmable logic devices

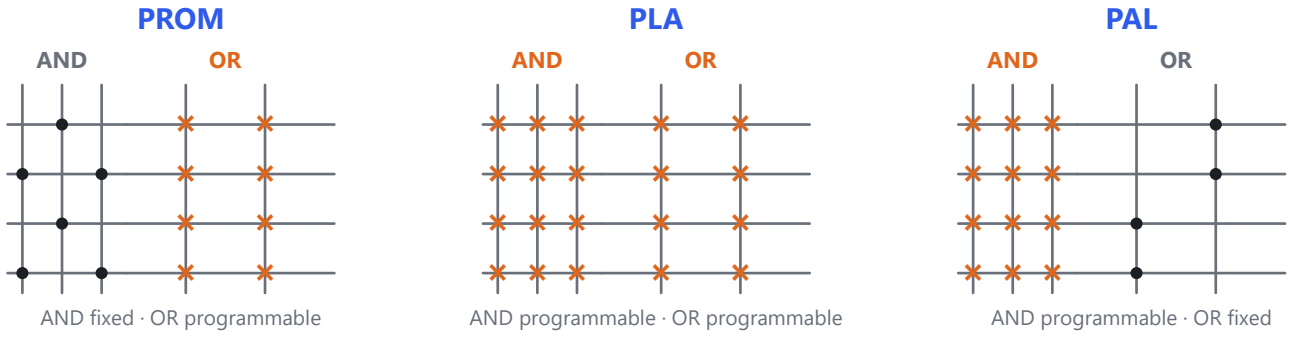
Anna Univ. EC6302 · Nov/Dec 2021 · Part A Q8 · 2 marks

What is programmable logic array? How it differs from ROM?

Anna Univ. EC1201 · Nov/Dec 2021 · Part A Q10 · 2 marks

Compare PAL and PLA devices.

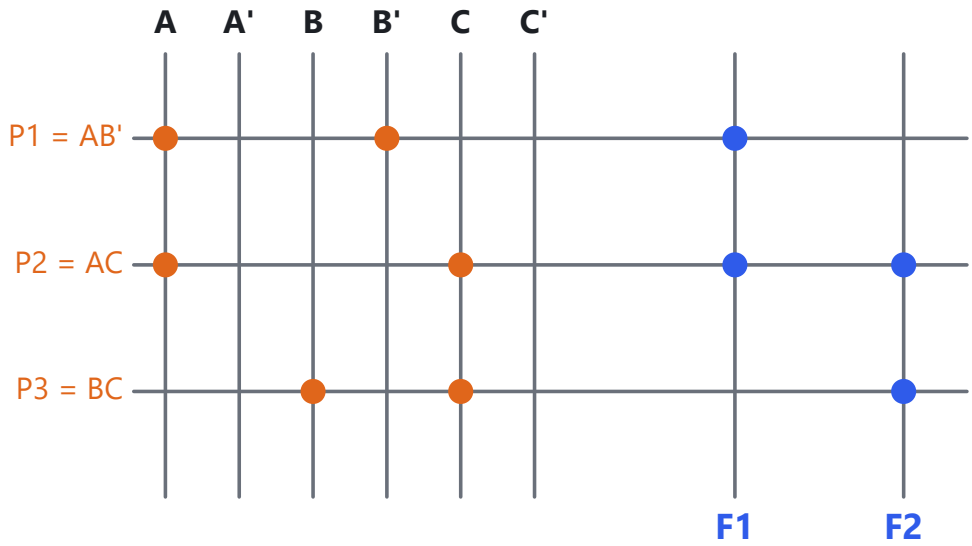
✎ **Guess first, on paper: every SOP has an AND level and an OR level. Which one would you make programmable?**



× = programmable connection ● = fixed connection

	AND array	OR array	Trait
PROM / ROM	fixed: a full decoder making every minterm	programmable	stores the whole truth table; no minimisation needed; grows as 2^n
PLA	programmable	programmable	most flexible; outputs can **share** product terms; slowest
PAL	programmable	fixed	each output gets its own fixed set of products; faster, cheaper, most common

Implementing on a PLA: minimise every output as SOP, look for products shared between outputs, then mark the connections. Example: $F_1 = \Sigma m(4,5,7) = AB' + AC$, $F_2 = \Sigma m(3,5,7) = BC + AC$. That needs 3 product terms, not 4, because AC is shared.



AC is generated once and shared by both outputs

Product	A B C	F_1 F_2
AB’	1 0 –	1 –
AC	1 – 1	1 1
BC	– 1 1	– 1

This is the PLA programming table that exam answers ask for: 1 = true input, 0 = complemented input, – = not connected. Beyond these: GAL (a re-programmable PAL), CPLD (many PALs on one chip), and FPGA (look-up tables, essentially small MUX-based ROMs, programmed in Verilog).

Answers. PLA: a PLD with programmable AND and OR arrays, so it generates only the product terms needed, and outputs can share them. A **ROM** has a fixed AND array that generates all 2^n minterms. **PAL vs PLA:** PAL's OR array is fixed, which makes it faster and cheaper but less flexible, with no sharing between outputs. PLA programs both arrays.

Check D7. (a) What size ROM implements a 4-bit × 4-bit multiplier? (b) Why is a PAL faster than a PLA?

Part D: answers to the Checks

1. **D1** min(8/0.4, 400/20) = min(20, 20) = **20**.
2. **D5** (a) NOR: PMOS in **series** (both inputs must be 0 to pull HIGH), NMOS in parallel. (b) $P \propto V^2 f \rightarrow (½)^2 \times 2 =$ **½**.
3. **D6** (a) 16K = 2^{14} → 14 address lines. 64K × 8 needs ×4 the words and ×2 the width, so **8 chips**. (b) EPROM is erased by UV light, the whole chip, out of circuit. EEPROM is erased electrically, byte by byte, in circuit.
4. **D7** (a) 8 address inputs (4 + 4 bits) → 2^8 = 256 words, and the product is up to 8 bits: **256 × 8**. (b) A fixed OR array has fewer programmable switches in the signal path, so there is less delay and capacitance.

MOCK

Practice paper: real questions only

This paper uses the MTE's three bands. The real ETE format is unknown. Allow about 3 minutes per mark. Close the pack and write full answers.

Section A · 2 marks each

1. Define Noise margin. Anna 2021 A2
2. What is the minimum no of flip flops needed to design a counter of modulus 60? Anna 2015 A6
3. What is edge-triggered flip-flop? Anna 2021 A6
4. Briefly explain about EEPROM. Anna 2021 A7
5. What is the most important consideration in making state assignments for asynchronous network? Anna 2015 A10 / 2021 A10
6. State the advantages of CMOS logic. Anna 2015 A2

Section B · 4–8 marks each

7. Discuss the working of a 4 bit Johnson counter with neat block diagram. Anna 2015 13(b)(i)
8. Draw and explain Tri-state TTL inverter circuit diagram with its operation. Anna 2021 11(b)(i)
9. Write the differences between static and dynamic RAM. Draw the circuits of one cell of each and explain its working. Anna 2021 14(a)
10. Give the hazard free realization for f (A, B, C, D) = ΠM (0, 2, 6, 7, 8, 10, 12). Anna 2021 15(b)
11. Explain how a state graph for a sequential machine can be converted to an equivalent ASM chart. Anna 2015 15(a)(i)

Section C · design

12. Design a sequential pattern detector that receives a stream of input bits. The circuit should recognize the pattern 010 and produce an output whenever this pattern is received. *(Moore, D flip-flops.)* Anna 2015 15(b)(ii)
13. Design a synchronous decade counter to count in the following sequence. 1, 0, 2, 3, 4, 8, 7, 6, 5. *(D flip-flops.)* Anna EC1201 2021 14(a)(i)

Mock: answer key

1. D1 definition + $NM_H = V_{OH} - V_{IH}$, $NM_L = V_{IL} - V_{OL}$.
2. 6 ($2^6 = 64 \geq 60 > 32$).
3. An FF that samples its inputs and changes state only at the clock's rising or falling **edge**, not during the whole level. This removes race-around and transparency.
4. Electrically erasable PROM: non-volatile, erased and re-written electrically, byte by byte, in circuit. Slower writes and limited write cycles.
5. Avoid **critical races**: give states that transition into each other codes that differ in one bit (adjacent assignment).
6. ≈ zero static power, large noise margin, high fan-out, rail-to-rail output, wide supply range, high density.
7. B3.
8. D3.
9. D6.
10. C7: f = B′D + C′D + A′BC′ + ABC + AD.
11. C5.
12. C2 Moore: $D_A = Bx + AB'x'$, $D_B = x'$, $z = AB$.
13. 4 D-FFs ($Q_3Q_2Q_1Q_0$), states 9–15 = X: $D_3 = Q_2Q_1'Q_0' \cdot D_2 = Q_1Q_0 + Q_2Q_1 + Q_3 \cdot D_1 = Q_2'Q_0' + Q_2Q_1Q_0 \cdot D_0 = Q_1Q_0' + Q_2Q_1'Q_0 + Q_3$. Self-start check: 9→5, 10→7, 11→5, 12→13→5, 13→5, 14→5, 15→7. Every unused state returns to the count, so the counter is **self-starting**.

Sources

- **Scope:** ECE 2105 syllabus slide (professor's deck 1) and the 2025 MTE paper, both in `exam-pack/`.
- **Facts:** prescribed texts (A. Anand Kumar 2e; R. P. Jain 4e; Brown & Vranesic 3e; Mano & Ciletti) via the subject knowledge base (stage-2✓, triangulated). TTL DC table checked against Cornell PHYS3360 L26 and datasheet summaries. ECL levels from Cornell L26 (−0.8 / −1.7 V typical).
- **Questions:** Anna University B.E. EC6302 Digital Electronics, Apr/May 2015 (QP 77112) and Nov/Dec 2021 (QP 70427); EC1201 Digital Electronics, Nov/Dec 2021 (QP 86565); RTU III-sem Digital Electronics (3E2075, 2012), via SKIT digital library. Items marked "practice" are not from a paper.
- **Images:** Wikimedia Commons (CMOS inverter, public domain; CMOS NAND, CC BY-SA 3.0; 6T SRAM cell, public domain). Everything else is generated by `figs.py`.